

Object-Oriented Programming (1)

COMP1039 Programming Paradigms

Yuan Cheng

Spring 2026

School of Computer Science
The University of Nottingham Ningbo China

Object-Oriented Programming

Procedural vs. Object-Oriented

Procedural programming (e.g., C, Pascal):

- Program = a **sequence of instructions**
- Focus: **what to do** step by step

Object-oriented programming (e.g., Java, Python):

- Program = **objects** that combine data + behavior
- Focus: **who owns the data** and **what they can do**

Example: Door (same idea, different style)

Procedural style:

```
1 unlockDoor(door);  
2 openDoor(door);  
3 closeDoor(door);  
4 lockDoor(door);
```

OOP style:

```
1 door.unlock();  
2 door.open();  
3 door.close();  
4 door.lock();
```

Question: Which style reads more like the real-world concept of a door?

Classes, Objects, Methods

- **Class**: a **template** (blueprint)
- **Object**: an **instance** of a class (a concrete thing)
- **Method**: a function **belonging to a class/object**

In OOP, you define the data and the routines that are permitted to act on that data.

- Code is organized around **data**.

Example: Elephant

Elephant (Class):

- **Data (fields):** name, color, height, weight, age, ...
- **Methods (behavior):** getColor(), getHeight(), moveForward(), ...

An instance of Elephant named bill (Object):

- **Data:** specific values assigned to each field (e.g., name = "Bill", age = 5, color = "Gray")
- **Methods:** all methods defined in the class can be called through this object

Key idea: A *class* is a blueprint; an *object* is a concrete instance.

Class: General Form

```
1  class ClassName {  
2  
3      // instance variables (attributes)  
4      Type varName;  
5  
6      // constructor  
7      ClassName(parameters) {  
8          // initialization code  
9      }  
10  
11     // methods  
12     ReturnType methodName(parameters)  
13         {  
14         // method body  
15     }
```

Instance Variables

- Attributes of the class
- Each object has its own copy

Constructor

- Called when an object is created
- Initializes object state

Methods

- Define behaviour
- Operate on instance data

Example Class: Elephant (data only)

```
1 class Elephant {  
2     String name; // name of the elephant  
3     String color; // color of the elephant  
4     int age;      // age of the elephant  
5 }
```

Creating Objects: reference variables

```
1 class ElephantDemo {  
2     public static void main(String[] args) {  
3  
4         Elephant bill;           // declaration (reference variable)  
5         bill = new Elephant(); // object creation on heap  
6  
7         Elephant andy = new Elephant();  
8     }  
9 }
```

- new allocates an object and returns a **reference**
- bill and andy refer to **different objects**

Dot operator: accessing members

```
1 Elephant bill = new Elephant();  
2 bill.age = 5;           // set instance variable  
3 System.out.println(bill.age);
```

Mini-check: What does `object.member` mean in Java?

Assignment: primitive types (predict)

```
1 int x = 6;
2 int y = 9;
3
4 x = y;    // copy the value of y into x
5 y = 3;    // change y afterwards
```

Question: What is the value of x now?

Assignment: primitive types

```
1 int x = 6;
2 int y = 9;
3
4 x = y;    // copy the value of y into x
5 y = 3;    // change y afterwards
```

Question: What is the value of `x` now?

```
x = y;
```

Primitives copy the **value**.

Assignment: reference types (predict)

```
1 Elephant bill = new Elephant();
2 bill.age = 10;
3
4 Elephant andy = new Elephant();
5 andy.age = 1;
6
7 andy = bill;      // reference assignment
8 andy.age = 5;     // modify through andy
```

Question: What is bill.age now?

Assignment: reference types

```
1 Elephant bill = new Elephant();
2 bill.age = 10;
3
4 Elephant andy = new Elephant();
5 andy.age = 1;
6
7 andy = bill;      // reference assignment
8 andy.age = 5;     // modify through andy
```

Question: What is bill.age now?

```
andy = bill;
```

References copy the **address/reference**, not the object.

- If no references point to an object, it becomes **eligible for garbage collection**
- Java manages memory automatically (no manual `free()` / `delete`)

Question: After `andy = bill`, what happens to the object that `andy` used to refer to?

Methods

What is a Method?

- Defines the **behavior** of a class
- Operates on the object's data (instance variables)
- Hides implementation details from the user

General Form:

```
1 returnType methodName(parameters) {  
2     // statements  
3     return value; // if returnType is not void  
4 }
```

Key Points:

- Parameters are local variables (scope: inside the method)
- The return type can be any valid type or void

Another Example: Vehicle class (data)

```
1 class Vehicle {  
2     int passengers; // max number of passengers  
3     int fuelCap;    // fuel capacity (gallons)  
4     int mpg;        // miles per gallon  
5 }
```

Why Methods?

Goal: compute $\text{range} = \text{fuelCap} * \text{mpg}$.

```
1 Vehicle car = new Vehicle();
2 car.mpg = 12;
3 car.fuelCap = 14;
4
5 int carRange = car.mpg * car.fuelCap;
6 System.out.println(carRange);
```

Question: What is the problem with calculating range like this everywhere?

Add a Method: encapsulate behavior

```
1 class Vehicle {  
2     int passengers;  
3     int fuelCap;  
4     int mpg;  
5  
6     int range() {  
7         return fuelCap * mpg;  
8     }  
9 }
```

```
1 Vehicle car = new Vehicle();  
2 car.mpg = 12;  
3 car.fuelCap = 14;  
4 System.out.println("range: " + car.range());
```

Key idea: callers need not know **how** range is computed.

Returning a Value

Using the return statement

Method Definition

```
1 int range() {  
2     return fuelCap * mpg;  
3 }
```

Calling the method

Inside main

```
1 int range = car.range();  
2 System.out.println("range: " + range);
```

Key Points

- The returned value must match the declared return type
- return immediately ends the method
- For void methods:
 - Either reach the closing brace }
 - Or use return; explicitly

Methods with Parameters

```
1 class Vehicle {  
2     int fuelCap;  
3     int mpg;  
4  
5     double fuelNeeded(int distance) {  
6         return (double) distance / mpg;  
7     }  
8 }
```

```
1 Vehicle car = new Vehicle();  
2 car.mpg = 12;  
3 System.out.println(car.fuelNeeded(120));
```

Question: Why do we cast to double here?

Why constructors?

- Initializing variables in `main` is:
 - hard to manage
 - error-prone
- A **constructor** initializes an object when it is created

Constructor (no parameters)

```
1  class Vehicle {  
2      int passengers;  
3      int fuelCap;  
4      int mpg;  
5  
6      Vehicle() {  
7          fuelCap = 14;  
8          mpg = 12;  
9          passengers = 4;  
10     }  
11 }
```

```
1  Vehicle car = new Vehicle();
```

Rule: constructor name must match class name.

Constructor with Parameters

```
1  class Vehicle {
2      int passengers;
3      int fuelCap;
4      int mpg;
5
6      Vehicle(int f, int m, int p) {
7          fuelCap = f;
8          mpg = m;
9          passengers = p;
10     }
11 }

```

```
1  Vehicle car = new Vehicle(14, 12, 4);

```

Prompt: Why is this more flexible than a fixed constructor?

A common bug (predict the output)

```
1  class ThisTest {
2      int a, b;
3      public void setData(int a, int b) {
4          a = a;
5          b = b;
6      }
7      public void showData() {
8          System.out.println("a=" + a);
9          System.out.println("b=" + b);
10     }
11     public static void main(String[] args) {
12         ThisTest tt = new ThisTest();
13         tt.setData(10, 20);
14         tt.showData();
15     }
16 }
```

Question: What gets printed? Why?

Fix with this

```
1 class ThisTest {
2     int a, b;
3
4     public void setData(int a, int b) {
5         this.a = a;
6         this.b = b;
7     }
8 }
```

- `this` refers to the **current object**
- Helps distinguish instance variables from parameters/local variables

Example: Using this

ThisTest.java

```
1  class ThisTest {
2      int a, b;
3      public void setData(int a, int b) {
4          this.a = a;
5          this.b = b;
6      }
7      public void showData() {
8          System.out.println("a=" + a);
9          System.out.println("b=" + b);
10     }
11     public static void main(String[] args) {
12         ThisTest tt = new ThisTest();
13         tt.setData(10, 20);
14         tt.showData();
15     }
16 }
```

Question: What is the output?

Practice

In-class Practice (5 minutes)

Task A: Define a class `Student` with fields:

- `name (String)`
- `id (int)`
- `gpa (double)`

Task B: Add:

- a constructor to initialize all fields
- a method `printInfo()` that prints one line

Stretch: What happens if you assign one `Student` reference to another?

- Take **1–2 minutes** to reflect on this lecture.
- On a sheet of paper, write:
 - **One thing you learned** in this lecture
 - **One thing you didn't understand**